# Predictive Memory Leak Detection Using Time Series Analysis



## 1. Introduction

Memory leaks occur when software retains allocated memory that is no longer needed. In long-running applications, repeated retention can reduce available memory and contribute to performance degradation or failure. However, increasing memory use can also reflect legitimate workload changes or caching, so utilization alone cannot confirm a leak. Prior work has explored detecting leak-like growth from memory-utilization time series without inspecting an application's internal objects [1].

Operating-system telemetry provides a basis for studying this behavior over time. Memory measurements describe resource availability; CPU activity, page faults, swap activity and disk throughput can provide workload context when available and informative. Forecasting uses past measurements to estimate a future memory value. Assi et al. compared Box-Jenkins and long short-term memory (LSTM) models for Windows performance forecasting [2]. Their comparison motivates evaluating an LSTM against simpler alternatives before attributing practical value to model complexity.

Accurate forecasting and useful leak warnings are distinct objectives. A model may predict most memory values closely while missing a brief period of sustained growth. A sensitive growth rule may identify injected leaks while also triggering during ordinary activity. Forecast error alone cannot establish detection quality. This study therefore asks whether improved memory forecasts translate into better warnings, assessed through recognition of labeled leak intervals and alarms outside those intervals.

The implemented pipeline forecasts a memory-pressure proxy defined as one minus the ratio of available memory to total physical memory. This quantity describes memory availability; it is not an estimated probability of a leak. Persistence, local linear extrapolation and a restricted autoregressive integrated moving average (ARIMA) baseline are compared with a small multivariate LSTM. Each recording is divided chronologically into training, validation and test periods. Learned preprocessing parameters are fitted on training data, and model and warning settings are selected using validation data. Forecasts are evaluated using mean absolute error and root mean squared error; warning behavior is assessed separately.

The pilot combines two complementary experimental settings without merging their timelines. The Antarex HPC Fault Dataset supplies real telemetry collected under controlled fault injection, including labeled memory-leak intervals [3]-[5]. Its CPU-MEM_mono block supports forecasting and leak-warning evaluation. Westermo provides industrial test-system telemetry [6]; systems 1-3 are used as separate forecasting experiments. Their leak status is unknown, so alarms on Westermo cannot be scored as confirmed true or false leak detections.

The study contributes a reproducible pilot comparison, a separate evaluation of trend-only and forecast-assisted warning rules, and a local prototype for replaying the verified experiments. Conclusions are limited to the observed workloads and available labels. The experiments measure recognition after leak injection begins; they do not establish warning before memory exhaustion or prevention of system failure.

## 2. Literature Survey

### 2.1 Software aging and resource exhaustion

Software aging research connects prolonged execution with accumulated internal errors and resource depletion. Huang et al. formalized software rejuvenation as preventive renewal of a running application [7]. This establishes that proactive action against aging is a longstanding research direction. It also separates two questions: whether degradation can be recognized and when an intervention should occur. A memory forecast alone answers neither question completely.

Vaidyanathan and Trivedi modeled resource exhaustion using both elapsed time and workload state, based on measurements from UNIX systems [8]. Grottke et al. subsequently examined resource usage in a web server under an artificial workload and considered the implications for rejuvenation [9]. These studies motivate including workload context when interpreting memory growth. They also caution against equating a trend observed under one workload with a universal aging rate. The present pilot retains informative CPU and memory-related measurements, while treating each recording as a separate experiment.

### 2.2 Runtime instrumentation and heap analysis

Detailed memory analysis can expose information that aggregate telemetry cannot. Nethercote and Seward describe Valgrind as a framework for dynamic binary instrumentation, including the shadow-state machinery used by analysis tools such as Memcheck [10]. Such instrumentation can support diagnosis close to program execution, whereas system-level memory measurements summarize the combined effects of many activities.

Mitchell and Sevitsky's LeakBot ranks suspicious Java data structures and tracks their evolution to support automated diagnosis [11]. Jump and McKinley's Cork associates growing heap structures with types by working alongside garbage collection [12]. Weninger et al. visualize the evolution of grouped heap objects through time-travel and timeline views in AntTracks [13]. These approaches help investigate where retained objects accumulate. Our telemetry pipeline does not collect object-reference graphs or allocation sites, so its warnings cannot identify the responsible object or source-code location.

Beneder et al. evaluated two measurement-based runtime algorithms on an embedded Linux room controller [14]. Their work is particularly relevant because it studies leak recognition using operating-system measurements on a deployed device. It also shows why measurements and thresholds require validation in the intended environment. Results from an HPC recording do not by themselves validate a warning rule on a Windows laptop or an embedded controller.

### 2.3 Telemetry-based detection and production services

Jindal et al. introduce Linear Backward Regression, Precog and their variants for detecting leaks from system memory utilization without internal application knowledge [1]. This is close to the observation level used in the present project, although our local trend rule is not a reproduction of those algorithms. The comparison of trend-only and forecast-assisted rules asks whether a learned forecast adds useful information to recent growth under a common evaluation protocol.

RESIN takes a broader production-service approach: it first identifies suspicious leaking entities, then selectively collects heap snapshots for diagnosis and applies mitigation [15]. Its staged design distinguishes detection from root-cause analysis and operational response. Our prototype addresses forecasting, warning evaluation and replay, without implementing comparable

diagnosis or automated mitigation. Similarly, the Antarex fault-injection setting [3]-[5] provides controlled labels, while Westermo's industrial measurements [6] provide workload diversity. These sources support complementary experiments, with different limits on what can be concluded about leaks.

### 2.4 Statistical, neural and hybrid forecasting

Box, Jenkins and Reinsel describe statistical time-series modeling through model identification, estimation and checking [16]. Hyndman and Athanasopoulos emphasize comparing forecasting methods against simple benchmarks, including forecasts that repeat the latest observation [17]. These foundations motivate the project's persistence and statistical baselines. Our ARIMA search is deliberately restricted, so its results characterize that implemented configuration rather than every possible ARIMA model. Local linear extrapolation provides an additional comparison that explicitly projects recent growth.

Hochreiter and Schmidhuber introduced LSTM to address difficulties in learning dependencies across extended sequences [18]. Gers et al. added an adaptive forget gate for continual prediction [19]. These mechanisms justify considering recurrent models for telemetry histories, but they do not guarantee that an LSTM will outperform a simple predictor on a particular horizon. Assi et al.'s Windows study directly compares Box-Jenkins and LSTM forecasting [2], supporting an empirical comparison rather than an assumed neural-model advantage.

Hybrid forecasting is a distinct design choice. Zhang combines ARIMA with an artificial neural network to model linear and nonlinear behavior [20]; this is an ARIMA-ANN method, not specifically an LSTM. Samal and Kumar examine an ARIMA-LSTM hybrid for software fault prediction using open-source software datasets [21]. Battisti et al.'s hLSTM-Aging combines moving-average and convolutional LSTM components for software-aging forecasts [22]. These works provide relevant alternatives, but their targets and settings differ from the present memory-pressure proxy. Our implementation evaluates ARIMA and LSTM separately; it does not implement either hybrid architecture.

Model complexity also needs a strong baseline. Zeng et al. demonstrate competitive simple linear models in long-term forecasting comparisons with Transformers [23]. Their benchmark is different from our short-horizon telemetry task, so its ranking cannot be transferred directly. Its methodological lesson is applicable: a more complex architecture should earn its place through a fair comparison on the chosen data, horizon and metrics.

### 2.5 Trend testing and warning rules

Mann's nonparametric trend work provides a foundation for rank-based assessment of ordered observations [24]. Sen's estimator uses pairwise slopes to estimate a trend robustly [25]. Hamed and Rao show that serial correlation affects the behavior of the Mann-Kendall test and propose a variance correction for autocorrelated data [26]. These distinctions matter for frequently sampled telemetry, where neighboring measurements are related. A positive slope is not itself a statistically significant trend, and significance would still not establish a leak's cause. The implemented warnings use local least-squares slopes, validation-selected thresholds and consecutive qualifying observations. Mann-Kendall testing and Sen's slope remain possible extensions, not components of the current experiments.

### 2.6 Temporal validation and data leakage

Tashman explains how holdout periods, forecast origins and model updating affect out-of-sample comparisons [27]. Bergmeir et al. show that ordinary cross-validation can be valid for particular autoregressive settings with uncorrelated model errors [28]. This qualified result should not be interpreted as blanket permission to randomly split overlapping telemetry windows. The present chronological split reflects the intended task of forecasting later observations from earlier data.

Albelali and Ahmed's preprint examines leakage associated with sequence construction and validation design in LSTM forecasting [29]. Kapoor and Narayanan document broader leakage problems in machine-learning-based research and argue for explicit reporting of data separation and scientific claims [30]. Together, these sources motivate documenting when splitting, window construction, scaling and model selection occur. In our pipeline, split boundaries are established before eligible windows are formed, learned scaling uses training data, and validation data guide model and warning choices. Test data are reserved for the reported comparison. This design supports an auditable pilot; it does not eliminate the need for further validation across workloads and repeated training runs.

### 2.7 Forecast error and leak-warning evaluation

Hyndman and Koehler discuss how forecast-error measures can behave differently and become misleading under unsuitable conditions [31]. In this project, MAE and RMSE describe errors in the forecast memory value. Expressing them in percentage points of total RAM makes their units explicit. They do not measure the probability that a warning is correct. For the classification task, Saito and Rehmsmeier explain why precision-recall analysis is informative when positive and negative cases are imbalanced [32]. This motivates reporting precision and recall alongside false-alarm behavior, rather than relying on overall classification accuracy.

Time-series events also occupy intervals. Tatbul et al. extend precision and recall to range-based anomalies and make domain preferences explicit [33]. Kim et al. show how point adjustment can inflate reported anomaly-detection performance [34]. The present evaluation therefore reports actual warning decisions at individual timestamps separately from event detection and delay. It does not replace all timestamps in a detected leak interval with successful predictions. Its event definition is project-specific rather than an implementation of Tatbul et al.'s complete scoring framework.

Wu and Keogh identify flaws in widely used anomaly benchmarks that can distort apparent progress [35]. Schmidl et al. compare diverse time-series anomaly detectors across many datasets, examining effectiveness, efficiency and robustness [36]. These studies reinforce the need for explicit dataset scope, strong comparisons and clear label semantics. They do not turn the unlabeled Westermo recordings into leak benchmarks. For Antarex, recognition after a recorded injection begins must likewise be distinguished from warning before memory exhaustion.

### 2.8 Implications for the present study

The reviewed work already establishes predictive aging analysis, telemetry-based leak detection, hybrid forecasting and staged production diagnosis. The present study builds on these directions through a reproducible pilot that evaluates forecasting and warning behavior separately. Its contribution is the documented comparison on the selected recordings and the examination of the trade-off between warning precision, coverage and timing. Broader superiority, deployment reliability and prevention of exhaustion require experiments beyond the current evidence.

## References


[1] A. Jindal, P. Staab, P. Kulkarni, J. Cardoso, M. Gerndt and V. Podolskiy, "Memory Leak Detection Algorithms in the Cloud-based Infrastructure," arXiv:2106.08938, 2021. https://doi.org/10.48550/arXiv.2106.08938.

[2] M. J. Assi, A. A. Fahad and B. Al-Sarray, "Time series forecast modeling for the Windows operating system performance using Box-Jenkins and LSTM models," International Journal of Nonlinear Analysis and Applications, vol. 13, no. 2, pp. 2121-2131, 2022. https://doi.org/10.22075/ijnaa.2022.26974.3505.

[3] A. Netti, Z. Kiziltan, O. Babaoglu, A. Sirbu, A. Bartolini and A. Borghesi, "Antarex HPC Fault Dataset," version 1.0, Zenodo, 2018. https://doi.org/10.5281/zenodo.2553224.

[4] A. Netti, Z. Kiziltan, O. Babaoglu, A. Sirbu, A. Bartolini and A. Borghesi, "FINJ: A Fault Injection Tool for HPC Systems," in Euro-Par 2018: Parallel Processing Workshops, Lecture Notes in Computer Science, vol. 11339, pp. 800-812, 2019. https://doi.org/10.1007/978-3-030-10549-5_62.

[5] A. Netti, Z. Kiziltan, O. Babaoglu, A. Sirbu, A. Bartolini and A. Borghesi, "Online Fault Classification in HPC Systems through Machine Learning," arXiv:1810.11208, 2018, revised 2019. https://doi.org/10.48550/arXiv.1810.11208.

[6] P. E. Strandberg and Y. Marklund, "The Westermo test system performance data set," arXiv:2311.14510, 2023. https://doi.org/10.48550/arXiv.2311.14510.

[7] Y. Huang, C. Kintala, N. Kolettis and N. D. Fulton, "Software rejuvenation: analysis, module and applications," in Proceedings of the 25th International Symposium on Fault-Tolerant Computing, pp. 381-390, 1995. https://doi.org/10.1109/FTCS.1995.466961.

[8] K. Vaidyanathan and K. S. Trivedi, "A measurement-based model for estimation of resource exhaustion in operational software systems," in Proceedings of the International Symposium on Software Reliability Engineering, pp. 84-93, 1999. Duke institutional publication record.

[9] M. Grottke, L. Li, K. Vaidyanathan and K. S. Trivedi, "Analysis of software aging in a Web server," IEEE Transactions on Reliability, vol. 55, no. 3, pp. 411-420, 2006. https://doi.org/10.1109/TR.2006.879609.

[10] N. Nethercote and J. Seward, "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation," in Proceedings of PLDI, pp. 89-100, 2007. https://doi.org/10.1145/1250734.1250746.

[11] N. Mitchell and G. Sevitsky, "LeakBot: An Automated and Lightweight Tool for Diagnosing Memory Leaks in Large Java Applications," in ECOOP 2003, Lecture Notes in Computer Science, vol. 2743, pp. 351-377, 2003. https://doi.org/10.1007/978-3-540-45070-2_16.

[12] M. Jump and K. S. McKinley, "Cork: Dynamic Memory Leak Detection for Garbage-Collected Languages," in Proceedings of POPL, pp. 31-38, 2007. https://doi.org/10.1145/1190216.1190224.

[13] M. Weninger, L. Makor and H. Moessenboeck, "Memory Leak Analysis using Time-Travel-based and Timeline-based Tree Evolution Visualizations," in Smart Tools and Apps for Graphics, pp. 63-75, 2020.
https://doi.org/10.2312/stag.20201241.

[14] R. Beneder, B. Glatz, M. Horauer and T. Rauscher, "Memory leak detection runtime-service for embedded Linux devices," in Proceedings of the IEEE Emerging Technology and Factory Automation Conference, pp. 1-6, 2014.
https://doi.org/10.1109/ETFA.2014.7005223.

[15] C. Lou, C. Chen, P. Huang, Y. Dang, S. Qin, X. Yang, X. Li, Q. Lin and M. Chintalapati, "RESIN: A Holistic Service for Dealing with Memory Leaks in Production Cloud Infrastructure," in Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation, pp. 109-125, 2022.
USENIX publication.

[16] G. E. P. Box, G. M. Jenkins and G. C. Reinsel, Time Series Analysis: Forecasting and Control, 4th ed. Wiley, 2008.
https://doi.org/10.1002/9781118619193.

[17] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. OTexts, 2021. Online edition accessed 14 September 2026.
https://otexts.com/fpp3/.

[18] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.
https://doi.org/10.1162/neco.1997.9.8.1735.

[19] F. A. Gers, J. Schmidhuber and F. Cummins, "Learning to Forget: Continual Prediction with LSTM," Neural Computation, vol. 12, no. 10, pp. 2451-2471, 2000.
https://doi.org/10.1162/089976600300015015.

[20] G. P. Zhang, "Time series forecasting using a hybrid ARIMA and neural network model," Neurocomputing, vol. 50, pp. 159-175, 2003.
https://doi.org/10.1016/S0925-2312(01)00702-0.

[21] U. Samal and A. Kumar, "Improving software reliability: a hybrid ARIMA-LSTM approach for fault prediction," International Journal of System Assurance Engineering and Management, vol. 16, pp. 1757-1769, 2025.
https://doi.org/10.1007/s13198-025-02743-2.

[22] F. Battisti, A. Silva, L. Pereira, T. Carvalho, J. Araujo, E. Choi, T. A. Nguyen and D. Min, "hLSTM-Aging: A Hybrid LSTM Model for Software Aging Forecast," Applied Sciences, vol. 12, no. 13, article 6412, 2022.
https://doi.org/10.3390/app12136412.

[23] A. Zeng, M. Chen, L. Zhang and Q. Xu, "Are Transformers Effective for Time Series Forecasting?," Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, no. 9, pp. 11121-11128, 2023.
https://doi.org/10.1609/aaai.v37i9.26317.

[24] H. B. Mann, "Nonparametric Tests Against Trend," Econometrica, vol. 13, no. 3, pp. 245-259, 1945.
https://doi.org/10.2307/1907187.

[25] P. K. Sen, "Estimates of the Regression Coefficient Based on Kendall's Tau," Journal of the American Statistical Association, vol. 63, no. 324, pp. 1379-1389, 1968.
https://doi.org/10.1080/01621459.1968.10480934.

[26] K. H. Hamed and A. R. Rao, "A modified Mann-Kendall trend test for autocorrelated data," Journal of Hydrology, vol. 204, nos. 1-4, pp. 182-196, 1998. https://doi.org/10.1016/S0022-1694(97)00125-X.

[27] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: an analysis and review," International Journal of Forecasting, vol. 16, no. 4, pp. 437-450, 2000. https://doi.org/10.1016/S0169-2070(00)00065-0.

[28] C. Bergmeir, R. J. Hyndman and B. Koo, "A note on the validity of cross-validation for evaluating autoregressive time series prediction," Computational Statistics & Data Analysis, vol. 120, pp. 70-83, 2018. https://doi.org/10.1016/j.csda.2017.11.003.

[29] S. Albelali and M. Ahmed, "Hidden Leaks in Time Series Forecasting: How Data Leakage Affects LSTM Evaluation Across Configurations and Validation Strategies," arXiv preprint arXiv:2512.06932, 2025. https://doi.org/10.48550/arXiv.2512.06932.

[30] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, article 100804, 2023. https://doi.org/10.1016/j.patter.2023.100804.

[31] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," International Journal of Forecasting, vol. 22, no. 4, pp. 679-688, 2006. https://doi.org/10.1016/j.ijforecast.2006.03.001.

[32] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," PLOS ONE, vol. 10, no. 3, article e0118432, 2015. https://doi.org/10.1371/journal.pone.0118432.

[33] N. Tatbul, T. J. Lee, S. Zdonik, M. Alam and J. Gottschlich, "Precision and Recall for Time Series," Advances in Neural Information Processing Systems, vol. 31, 2018. NeurIPS publication.

[34] S. Kim, K. Choi, H.-S. Choi, B. Lee and S. Yoon, "Towards a Rigorous Evaluation of Time-Series Anomaly Detection," Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, no. 7, pp. 7194-7201, 2022. https://doi.org/10.1609/aaai.v36i7.20680.

[35] R. Wu and E. Keogh, "Current Time Series Anomaly Detection Benchmarks are Flawed and are Creating the Illusion of Progress," IEEE Transactions on Knowledge and Data Engineering, vol. 35, no. 3, pp. 2421-2429, 2023; first published online 2021. https://doi.org/10.1109/TKDE.2021.3112126.

[36] S. Schmidl, P. Wenig and T. Papenbrock, "Anomaly Detection in Time Series: A Comprehensive Evaluation," Proceedings of the VLDB Endowment, vol. 15, no. 9, pp. 1779-1797, 2022. https://doi.org/10.14778/3538598.3538602.